

Low-Level Design

RAG-Based Customer Support Assistant

Module Design, Data Structures & Interface Specifications

1. Module-Level Design

The system is composed of the following modules, each with a distinct responsibility within the RAG pipeline:

#	Module	Responsibility
a	Document Processing	Handles loading of the PDF file and extraction of raw text. Prepares the document for further preprocessing.
b	Chunking Module	Splits extracted text into smaller chunks (500–1000 tokens) for efficient retrieval and better context handling.
c	Embedding Module	Converts each text chunk into vector embeddings using an embedding model to capture semantic meaning.
d	Vector Storage Module	Stores embeddings in ChromaDB with indexing for fast similarity search.
e	Retrieval Module	Fetches top-k relevant chunks based on similarity between query embedding and stored vectors.
f	Query Processing Module	Processes user input, cleans it if needed, and converts it into embedding format for retrieval.
g	Graph Execution Module	Executes workflow using LangGraph, managing transitions between nodes.
h	HITL Module	Handles escalation to human agents when system confidence is low or query is complex.

2. Data Structures

a) Document Representation

The input document is represented as raw text extracted from the PDF. It may include metadata such as document name, page number, and source.

b) Chunk Structure

Each text chunk is represented with the following fields:

```
{
  chunk_id   : <unique identifier>
  text       : <content of the chunk>
  metadata   : {
    source    : <document name>
    page_number : <page number>
  }
}
```

c) Embedding Structure

Each chunk is converted into a vector embedding and stored with a reference:

```
{
  embedding_vector : [float, float, ...] // Numerical vector
  representation
  chunk_id        : <reference to original chunk>
}
```

d) Query Representation

User queries are represented as:

```
{
  query_text      : <raw user input>
  query_embedding : [float, float, ...] // Vector representation of query
}
```

e) Query-Response Schema

The system response structure includes:

```
{
  query           : <user input>
  retrieved_chunks : [chunk_1, chunk_2, ...] // Relevant context
  generated_answer : <LLM output>
  confidence_score : float // Optional evaluation metric
}
```

f) Workflow State Object

The LangGraph state object flows between nodes and maintains:

```
{
  input_query      : <user query>
  retrieved_context : [chunk_1, chunk_2, ...]
  llm_response     : <generated response>
  decision_flag    : 'answer' | 'escalate'
}
```

3. Workflow Design

a) Nodes

The LangGraph workflow consists of the following nodes:

Node	Description
Processing Node	Handles query processing, retrieval of relevant chunks, and response generation using the LLM.
Output Node	Responsible for delivering the final response to the user or triggering escalation to the HITL module.

b) Edges

Edges define the directed flow between nodes:

- Input → Processing Node
- Processing Node → Output Node

Conditional edges route based on the `decision_flag` in the workflow state: 'answer' routes to user delivery; 'escalate' routes to the HITL module.

c) State Management

A shared state object is passed between nodes. It contains `input_query`, `retrieved_context`, `llm_response`, and `decision_flag` — enabling seamless data transfer and decision-making across nodes.

4. Conditional Routing Logic

a) Answer Generation Criteria

The system delivers the response directly to the user when:

- Retrieved context is relevant to the query
- The generated answer is complete and coherent
- Confidence score meets or exceeds the defined threshold

b) Escalation Criteria

Escalation to HITL is triggered under any of these conditions:

- Confidence score is below threshold
- Retrieved context is insufficient or irrelevant
- User query is ambiguous, complex, or outside the knowledge base

c) Confidence Evaluation Methods

Confidence is estimated using:

- Similarity score from retrieved chunks (primary signal)
- LLM response validation (optional)
- Heuristic rules — e.g., short or vague answers indicate low confidence

d) Decision Logic

```
IF confidence_score >= threshold:  
    decision_flag = 'answer' --> Deliver response to user  
  
ELSE (confidence_score < threshold):  
    decision_flag = 'escalate' --> Route to HITL module
```

5. HITL (Human-in-the-Loop) Design

a) Escalation Trigger

The system escalates to a human agent when:

- Confidence score is below the defined threshold
- Retrieved context is insufficient or irrelevant
- Query is complex, ambiguous, or outside the knowledge base

b) Escalation Process

Once escalation is triggered, the following sequence occurs:

- The query and any partial system response are forwarded to a human agent
- The query is placed in a queue or notification system
- The human agent reviews the query and provides a response

c) Human Response Integration

The response provided by the human agent is:

- Sent back to the user as the final answer
- Optionally stored for future reference or system improvement

d) System Behavior During Escalation

While awaiting human input, the system:

- Notifies the user that the query is under review
- Displays a fallback message: "Your query is being reviewed by our support team"

6. API / Interface Design

a) Input Format

The system accepts user queries in JSON format:

```
// Request
{
  "query": "What is the refund policy?"
}
```

b) Output Format — Successful Response

```
// Response (answered)
{
  "query"      : "What is the refund policy?",
  "answer"     : "The refund policy allows returns within 30 days...",
  "source"     : "Document reference",
  "confidence" : 0.85,
  "status"     : "answered"
}
```

c) Escalation Response Format

```
// Response (escalated)
{
  "query"      : "Complex query...",
  "status"     : "escalated",
  "message"    : "Your query has been forwarded to a human agent."
}
```

d) Interaction Flow

- User submits query to the system
- System processes query through retrieval and LLM generation
- Decision logic evaluates confidence and determines response path
- Final output is returned to the user (directly or via human agent)

7. Error Handling

Error Condition	Handling Strategy
Invalid Input	Return a clear validation message to the user.
No Relevant Chunks Found	Fall back to escalation via HITL module.
LLM / API Failure	Retry once; if failure persists, return safe fallback message.
Retrieval / Embedding Error	Log the error and return a safe message to the user.
General System Errors	Return a generic error response; ensure no system crash.